

Study Guide & Plan

Governance, Risk, Security & Software-Engineering Topics — mapped to your graduation project

Project: Cyber-Physical Smart Building Access Control System | Prepared: 10 August 2026

A note on two ambiguous items from your list: "Certi bank" is interpreted here as the Central Bank of Egypt (CBE) Financial Cybersecurity Framework, and "Sdm" as "Secure/System Development Methodology" (a governance term covering secure SDLC). Both fit the theme of the rest of your list — if you meant something else (e.g. a specific course or certification name), tell me and I'll correct this section.

Part 1 — What Each Topic Means

Grouped into two clusters: Governance/Risk/Security topics, and Software-Engineering topics. Each includes a plain-language definition and how (or whether) it connects to your access-control project, based on your Project Proposal, System Design Document, and the backend/dashboard code I reviewed.

Cluster A — Governance, Risk, Compliance & Security

1. Governance

Definition: IT/security governance is the framework of policies, roles and decision rights that keeps technology and security work aligned with an organization's objectives and risk appetite — who decides what, who is accountable, and how oversight happens (e.g. via COBIT, ISO/IEC 27001, or the NIST Cybersecurity Framework's "Govern" function).

Project relevance: Your system already has a small governance model built in: the admin vs. instructor role split, who can trigger overrides, and how the university's access policy gets enforced through the dashboard. Section 6 of your proposal (security & fail-safe design) is effectively a governance decision — who is trusted to override what, and under which conditions.

2. How to Build KPIs

Definition: A Key Performance Indicator is a measurable value tied to a specific objective — chosen so it's quantifiable, has a target/threshold, and is reviewed on a set cadence. Good KPI design starts from the objective, not from whatever data happens to be available, and mixes leading indicators (predict problems) with lagging ones (confirm outcomes).

Project relevance: Section 13 of your proposal (Test & Evaluation Plan) is already a KPI set: unlock latency under 1 second, wired→Wi-Fi failover time, biometric false accept/reject rate, tamper-alert response time. You can extend this into an operational dashboard KPI panel — e.g. % of scheduled unlocks executed correctly, mean time to detect a forced-open door, node uptime — strong material for your results/evaluation chapter.

3. "Certi Bank" *(interpreted — see note above)*

Definition: Read as the Central Bank of Egypt's (CBE) Financial Cybersecurity Framework — the regulatory framework Egyptian banks and financial institutions must comply with, covering governance, risk management, secure development, access control, and incident response.

Project relevance: Indirect. Your project isn't a bank system, so CBE compliance itself doesn't apply — but the same control categories your proposal already implements (dual-factor access control, encrypted credentials, audit logging, tamper detection) are exactly what a CBE-style IT auditor checks for. Framing your security chapter against a named, recognized framework rather than ad hoc requirements strengthens it academically.

4. CRISC ("Crisk")

Definition: CRISC — Certified in Risk and Information Systems Control — is an ISACA certification focused on identifying, assessing, responding to and monitoring IT risk, and connecting that risk work to enterprise governance.

Project relevance: Direct. Section 12 (Risk Assessment & Mitigation) in your proposal is a CRISC-style exercise in miniature. Restructuring it explicitly as identify → assess → respond → monitor (e.g. risk: cloned RFID card → assessment: likelihood/impact → response: DESFire + fingerprint dual-factor → monitor: failed-auth alerting) will read as more rigorous to your committee.

5. "SDM" (*interpreted as Secure/System Development Methodology — see note above*)

Definition: Most governance frameworks (COBIT, ISO 27001 Annex A.8.25–28, and CBE's framework) require a documented Secure/System Development Methodology: a defined SDLC with security checkpoints — requirements → secure design → secure coding → security testing → deployment → maintenance — rather than ad hoc development.

Project relevance: Direct — and already something you've done, not something to add. Your own project structure (Phase 1 design doc → Phase 2 wiring/bench test → Phase 3–4 backend/dashboard → Phase 5 dedicated security review → Phase 6 deployment → Phase 7 system test report) is a working example of an applied secure SDLC. You can cite it as exactly that in your methodology chapter.

6. Data Anonymizing

Definition: Removing or transforming personally identifiable information so individuals can't be re-identified — via masking, pseudonymization, generalization, k-anonymity, or differential privacy — typically so data can be used for testing, analytics or demonstration without exposing real people's information.

Project relevance: Direct. Your access_events log ties fingerprint templates, card UIDs and timestamps to real students/instructors — sensitive personal data. If you show sample logs, screenshots, or exported data in your report or defense demo, anonymize names/card IDs (use "Student A", synthetic UIDs) rather than real seeded accounts. Worth a short paragraph in your report showing you considered this.

7. AI Guardrails

Definition: The technical and policy controls placed around an AI system (typically an LLM) to keep its outputs within safe, intended bounds — input/output filtering, permission scoping, rate limiting, human-in-the-loop approval for risky actions, and monitoring for prompt injection or misuse.

Project relevance: Not related to the current proposal — there's no AI/LLM component described in it. Only becomes relevant if you later add an AI feature (e.g. anomaly detection on access patterns, or a support chatbot on the dashboard); guardrails would then apply narrowly to that feature, not to the door-control system itself.

8. Archer (RSA Archer)

Definition: RSA Archer is a commercial GRC (Governance, Risk & Compliance) platform used by large organizations to centralize risk registers, compliance workflows, audit tracking and policy management.

Project relevance: Not related to your build. It sits in a different product category from your project's competitors (Section 8 already names HID, Salto, Genetec — physical access control vendors; Archer is enterprise compliance-management software). Good to recognize by name for interviews or a GRC-track career, not something to reference in the project itself.

9. Shellbags

Definition: Windows registry artifacts (stored in NTUSER.DAT/USRCLASS.DAT) that record a user's folder-browsing history — including deleted or removable folders — making them a key artifact in Windows digital forensics investigations.

Project relevance: Not related. Your system is an embedded/web stack (ESP32 + Python/FastAPI + React), not a Windows forensics target. This is a standalone DFIR (digital forensics & incident response) skill — useful if you're building toward a security/forensics career track, unrelated to this project.

10. PowerShell

Definition: Microsoft's command-line shell and scripting language, used for Windows/system administration and automation — and, because of that power, also a very common post-exploitation tool for attackers, which is why defenders study it too.

Project relevance: Minor/indirect. Your stack doesn't touch Windows (Python backend, React/Vite dashboard, ESP32 firmware). It would only matter if the dashboard or backend were ever administered from Windows servers — not part of your current architecture. Treat it as a general sysadmin/security skill outside the project's scope.

Cluster B — Software Engineering

11. Use Case (Software Engineering)

Definition: A use case describes an interaction between an actor (a user or another system) and the system under design, to reach a goal: actor, preconditions, main success scenario, alternate/exception flows, and postconditions. A standard requirements tool, often paired with UML use-case diagrams.

Project relevance: Direct. Your actors are already implicit in the proposal: Admin, Instructor, Door Node, Registrar system. Example: "Instructor overrides a door lock outside the timetable" — precondition: authenticated instructor, node online; main flow: dashboard → backend → MQTT command → node unlocks → event logged; exceptions: node offline, invalid session. Writing 4–6 of these formally strengthens your requirements chapter and gives you clean material for a use-case diagram.

12. Design Patterns

Definition: Reusable, named solutions to recurring software design problems (Observer, Factory, Strategy, Singleton, Repository, and others) that improve maintainability and let engineers communicate design intent quickly.

Project relevance: Direct. Your MQTT event listener (`app/services/mqtt_service.py`, subscribing to `site/+status`, `.../event`, `.../alert`) is a natural fit for the Observer pattern — subscribers reacting to door events. Your SQLAlchemy data-access layer maps well to a Repository pattern. Naming the pattern(s) you actually used — even if you didn't set out to — in your System Design Document adds credibility with a technical committee.

13. Waterfall

Definition: A linear SDLC model — requirements → design → implementation → testing → deployment → maintenance — where each phase is completed and (often formally) signed off before the next begins.

Project relevance: Direct — this is literally how your project is organized: a fixed 34-week plan across 7 numbered phases, each producing a document (System Design Document, Wiring & Bench Test Guide, Security Review, Multi-Node Deployment Guide, System Test Report). Name this explicitly as your chosen methodology in the report rather than leaving it implicit.

14. Agile

Definition: An iterative SDLC approach (Scrum, Kanban, etc.) built around short cycles, a continuously re-prioritized backlog, incremental working software, and adapting to feedback rather than following a fixed upfront plan.

Project relevance: Indirect — its main value here is as the comparison point that justifies your Waterfall choice. Examiners commonly ask "why this methodology?" Your answer: hardware lead times, safety-critical fail-secure/fail-safe requirements needing signed-off design before fabrication, and a fixed defense deadline all favor a phased plan over Agile's continuous re-prioritization. Have this comparison ready.

15. Node Server

Definition: Node.js is a JavaScript runtime for running JS outside the browser — commonly used to build backend servers (Express, Fastify), or, as in your case, to run a frontend's build/dev tooling.

Project relevance: Partial. Your dashboard folder (package.json, vite.config.js, React 19) confirms Node.js is present as the toolchain running Vite's dev/build server for the dashboard — not as your backend. Your actual backend is Python/FastAPI + SQLAlchemy + paho-mqtt (confirmed from requirements.txt). Know this distinction precisely: if asked "is this a Node backend?" in your defense, the correct answer is no — it's Python, Node only runs the frontend tooling.

Part 2 — Study Plan (5 Weeks)

Ordered so the topics that directly strengthen your graduation report come first, and general/career topics come last. Adjust the pace to however long you actually have before your defense — the order matters more than the exact week count.

Week 1 — Software Engineering Foundations

Goal: Turn your existing project into properly-labeled academic material.

- Learn use-case structure (actor, pre/postconditions, main + exception flows); write 4–6 use cases for your system.
- Review Observer, Repository and Factory patterns; identify where your codebase already uses pattern-like structures.
- Compare Waterfall vs. Agile properly — be ready to justify your methodology choice in one paragraph.
- Clarify the Node.js vs. Python-backend distinction in your own stack so you can state it precisely if asked.

Tie back to the report: *Draft the "Requirements / Use Cases" and "Methodology" sections of your report using your own project as the worked example.*

Week 2 — Governance, Risk & KPIs

Goal: Reframe your existing security/evaluation sections using formal frameworks.

- Study basic governance models (who decides, who's accountable, oversight cadence).
- Learn KPI design: pick objective → pick indicator → define measurement → set target/threshold.
- Read the CRISC risk lifecycle (identify → assess → respond → monitor) and re-map your Section 12 risks to it.
- Understand "secure SDLC" / SDM concepts and recognize your own 7-phase plan as an instance of one.

Tie back to the report: *Rewrite Section 12 (Risk Assessment) and Section 13 (Test & Evaluation) using the risk-lifecycle and KPI structures above.*

Week 3 — Data Protection & Compliance Context

Goal: Add a short, credible data-handling narrative to your report.

- Learn core anonymization techniques (masking, pseudonymization, k-anonymity) and apply one to any sample data you show.
- Skim the CBE Financial Cybersecurity Framework's control categories (governance, access control, secure development, incident response) as a named reference framework — confirm with me if "Certi bank" meant something else.
- Get a conceptual (not hands-on) overview of what GRC platforms like Archer do, for context and interview readiness.

Tie back to the report: *Add a short data-handling/privacy paragraph to Section 6, showing your logs/biometric templates are protected and any demo data is anonymized.*

Week 4 — Broader Security Skills (career track, not project-required)

Goal: Build general security literacy that isn't tied to this specific project.

- PowerShell basics: core cmdlets, how it's used defensively and offensively.
- Windows forensics fundamentals, including Shellbags, as one artifact among several.
- AI guardrails concepts — useful general knowledge, and relevant only if you extend the project with an AI feature later.

Tie back to the report: *No direct tie-in — this week is portfolio-building for internships/roles after graduation, not report content.*

Week 5 — Consolidation & Defense Prep

Goal: Rehearse, don't learn anything new.

- Read your report end-to-end; confirm every claim above (patterns, methodology, risk framing) matches what the code actually does.
- Prepare answers for the two "why" questions examiners ask most: why Waterfall not Agile, and why fail-secure vs. fail-safe per door.
- Walk through your risk table and KPI/evaluation table out loud from memory.
- Do a full run-through of your demo, including the anomaly/tamper-alert scenario.

Part 3 — Quick Relevance Summary

At a glance: what's worth your time for the project itself versus what's a separate skill/career track.

Topic	Relevance to your project	Why
Governance	Indirect	Admin/instructor role split & access policy = a small governance model
How to build KPIs	Direct	Your Test & Evaluation Plan (Sec. 13) is already a KPI set
"Certi Bank" (CBE framework, assumed)	Indirect	Same control categories (access, audit, encryption) as a bank's IT audit
CRISC / risk management	Direct	Sec. 12 Risk Assessment maps to CRISC's risk lifecycle
"SDM" (secure dev methodology, assumed)	Direct	Your 7-phase plan with a security review phase IS an applied SDM
Data anonymizing	Direct	Access logs contain real card/fingerprint/identity data — anonymize before sharing
AI guardrails	Not related (yet)	No AI/LLM component in the current proposal
Archer (RSA Archer GRC)	Not related	Enterprise GRC software; different category from your embedded system
Shellbags	Not related	Windows forensics artifact; unrelated to embedded/web stack
PowerShell	Not related	Your stack is Python/FastAPI + React/Vite + ESP32, not Windows admin
Use case (software engineering)	Direct	Formalizes your actors & flows (admin, instructor, door node, registrar)
Design patterns	Direct	MQTT listener ≈ Observer pattern; SQLAlchemy layer ≈ Repository pattern
Waterfall	Direct	Your 34-week, 7-phase plan is a textbook Waterfall structure
Agile	Indirect	Useful as the comparison you cite to justify choosing Waterfall
Node server	Partial	Node/Vite runs your dashboard's build tooling; backend is Python, not Node

Sources used to verify current facts (CRISC, RSA Archer, CBE framework, Shellbags, AI guardrails):

isaca.org/credentialing/crisc · cbe.org.eg/en/cybersecurity · tripwire.com (Egypt Financial Cybersecurity Framework) · sans.org (Windows Shellbags) · magnetforensics.com · infosectrain.com (RSA Archer)